

GNN GCN GAT MMPN

CHAE WON PARK

**Network Science Lab
Dept. of Artificial Intelligence
The Catholic University of Korea**

E-mail: codnjsld@catholic.ac.kr

Contents



- **GNN**
 - spectral & spatial
 - Neighborhood Aggregation

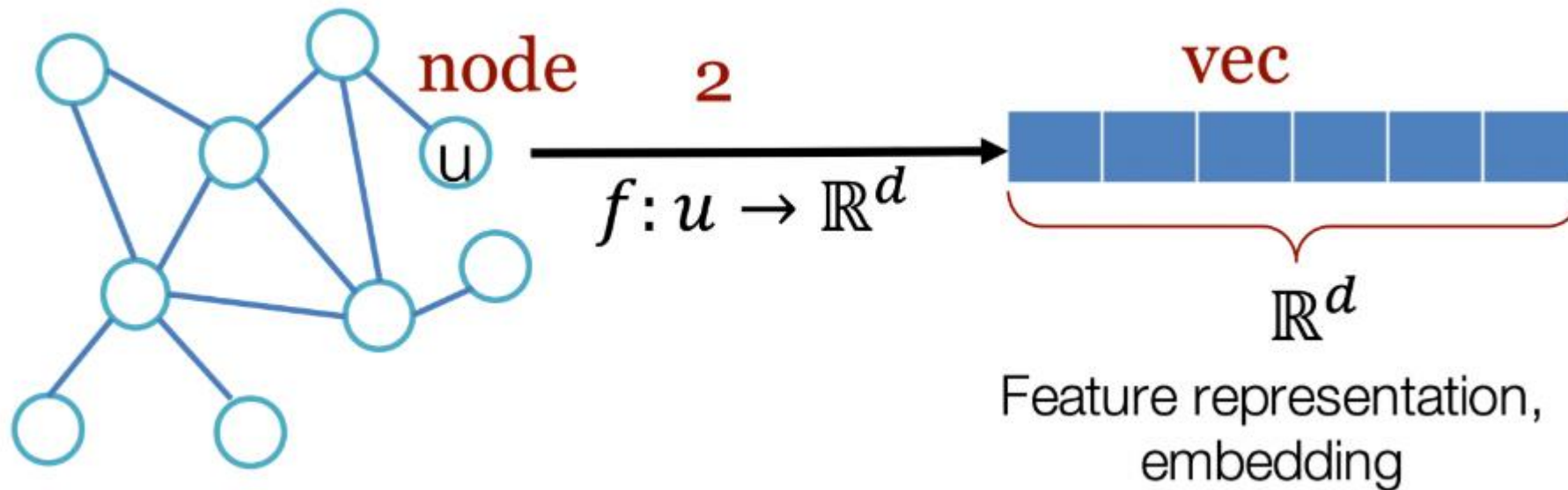
- **GCN**
 - Graph Laplacian Regularization
 - Mechanism
 - Experiments

- **GAT**
 - Graph Attention Layer
 - Multi-head attention
 - Experiments

- **MPNN**
 - Mechanism
 - Experiments

Graph Neural Network

- 비정형 데이터인 그래프를 입력으로 받아, 각 노드/그래프 전체의 특징을 벡터로 추출하는 신경망
- 관계와 특징을 모두 고려함 (edge, node)

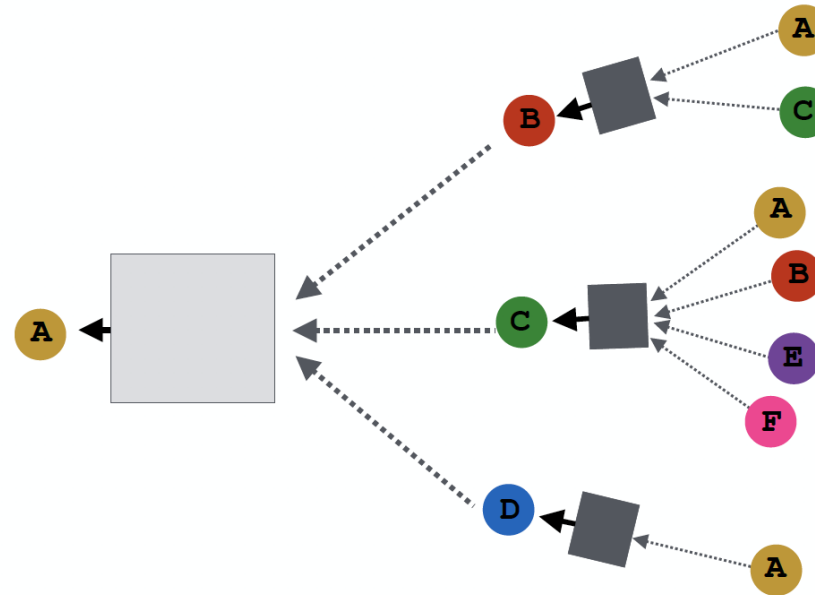
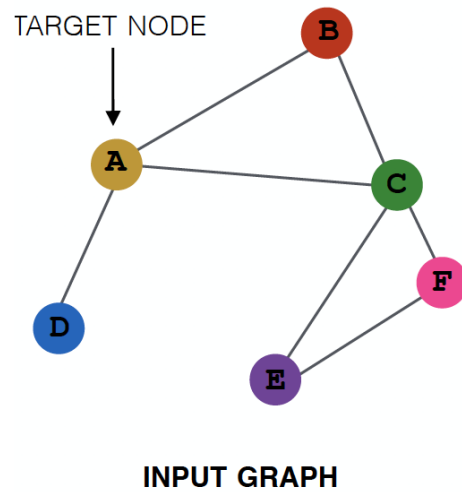


방법론

spectral	spatial
그래프의 라플라시안 행렬을 찾아 고유값 분해로 그래프의 latent 정보를 파악	가까운 이웃과 먼 이웃을 구분하여 이웃의 정보를 집계하여 convolution 연산을 수행
전체 그래프를 동시에 처리해야 해서 노드와 엣지가 거대한 그래프일수록 비실용적	효율성이 좋음
새로운 노드가 추가되거나 그래프가 바뀌면 고유벡터부터 새로 계산해야 함	그래프가 변해도 해당 노드의 이웃에 대해서만 새로 연산하면 됨
무방향 그래프만	모든 방향성 가능

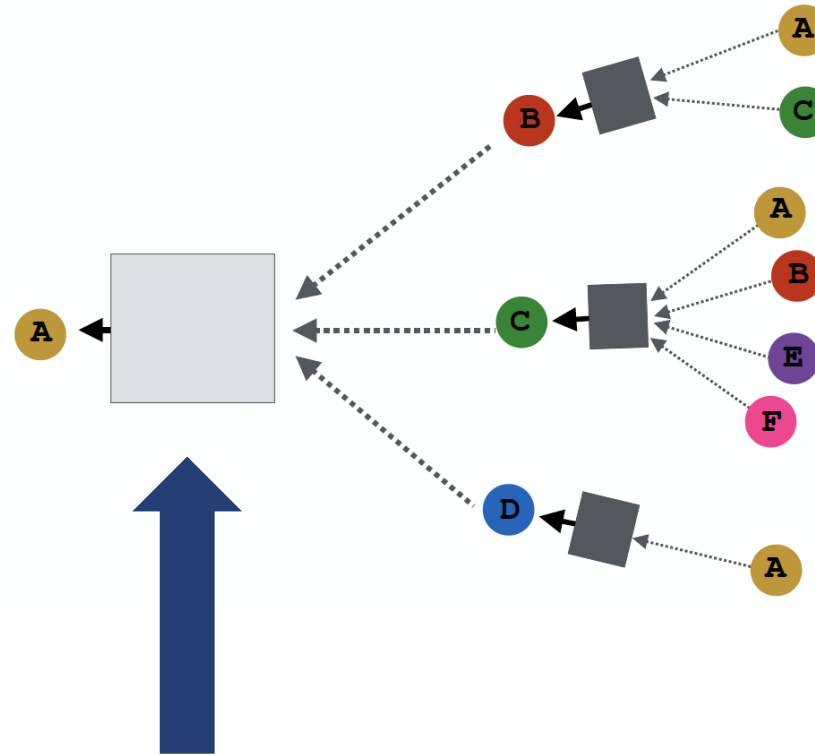
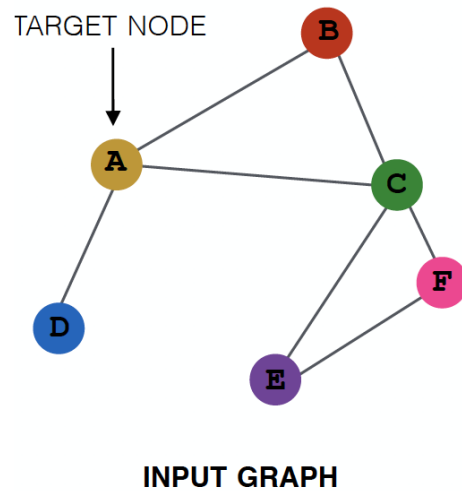
Neighborhood Aggregation

- Spatial
- GNN이 정보를 업데이트하는 과정



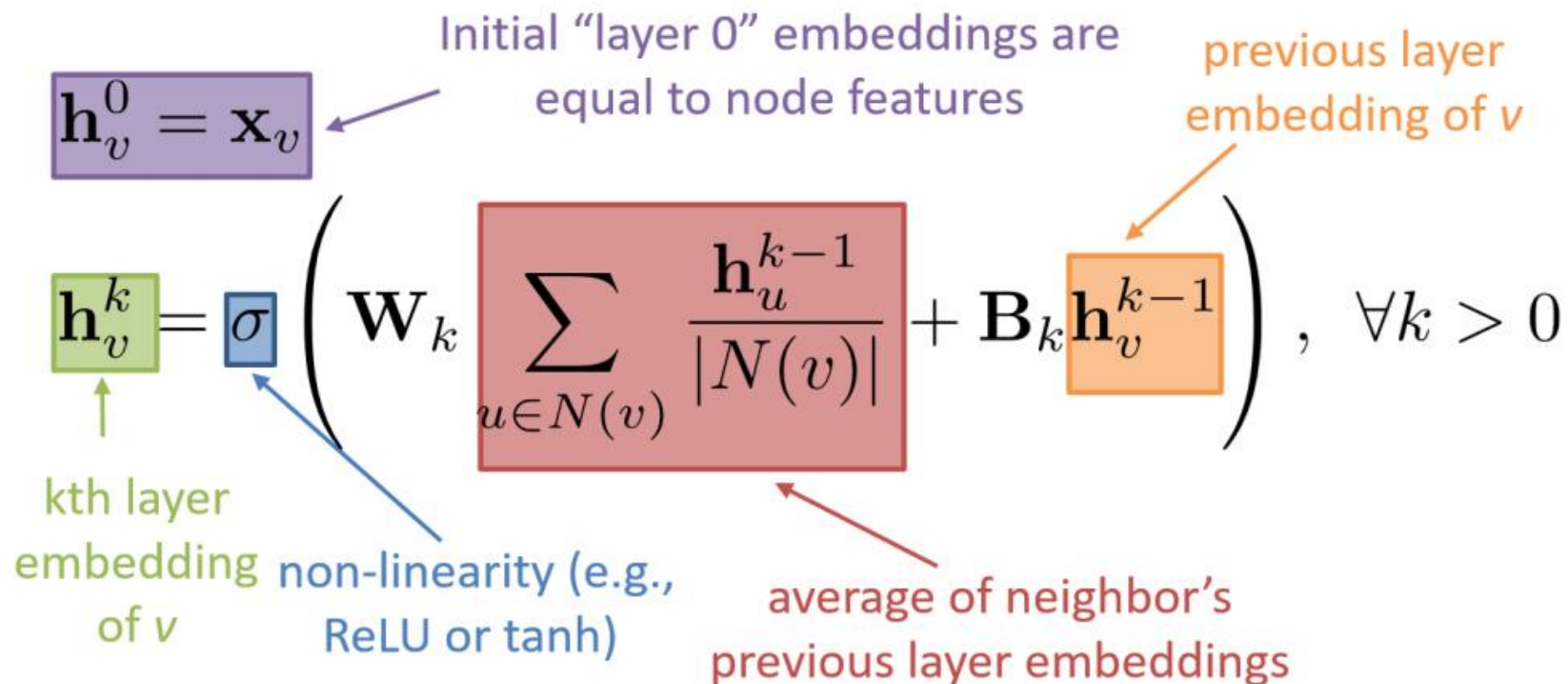
Neighborhood Aggregation

- Spatial
- GNN이 정보를 업데이트하는 과정



Aggregate function

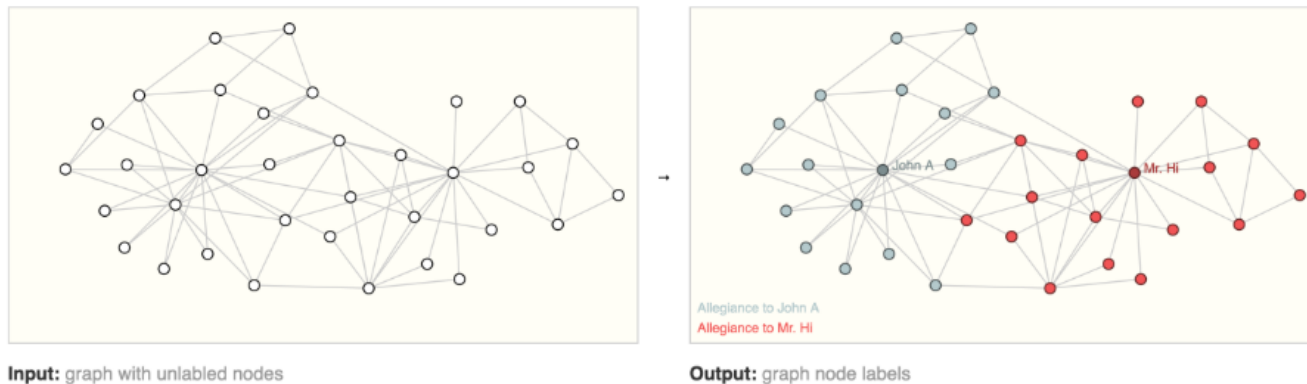
1. Aggregate : 이웃 노드들의 특징 벡터를 수집
2. Combine : 집계된 정보와 현재 노드 자신의 정보를 합침
3. Update : 합쳐진 정보를 다음 층의 새로운 노드 벡터로 갱신



Output Step

- Label 예측
- Task에 따른 구분

Node Classification



임의의 한 정점이 어떤 레이블에 속하는지 분류
Readout X

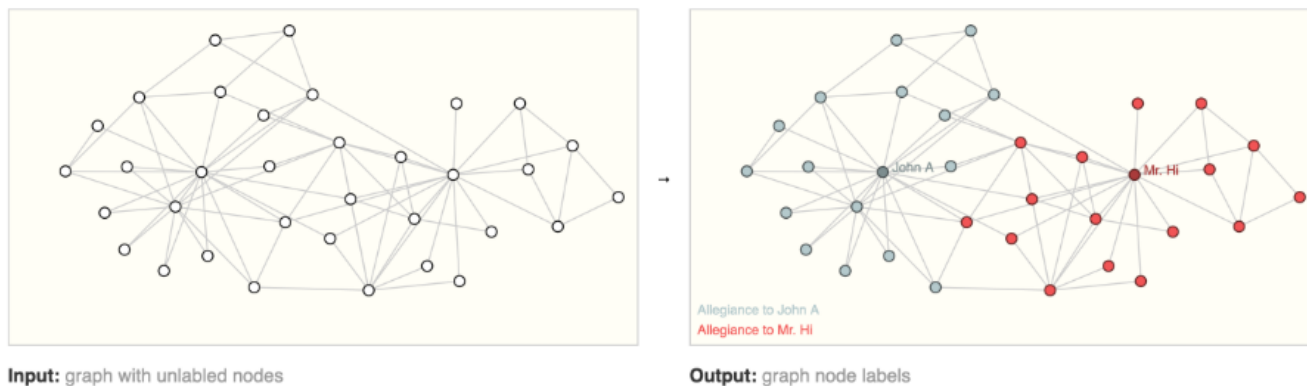
[출처] <https://distill.pub/2021/gnn-intro>, A Gentle Introduction to Graph Neural Networks

$$\text{softmax}(z_i)$$

Output Step

- Label 예측
- Task에 따른 구분

Node Classification



[출처] <https://distill.pub/2021/gnn-intro>, A Gentle Introduction to Graph Neural Networks

$$\text{softmax}(z_i)$$

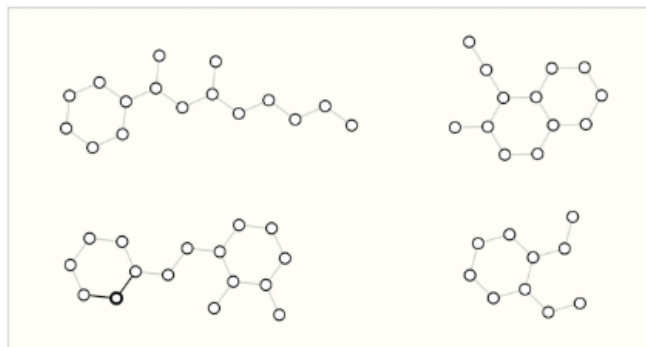
임의의 한 정점이 어떤 레이블에 속하는지 분류
Readout X

Readout : graph convolution layer을 통해 생성된 latent feature matrix를 그래프 전체를 표현하는 하나의 벡터로 변환하는 함수

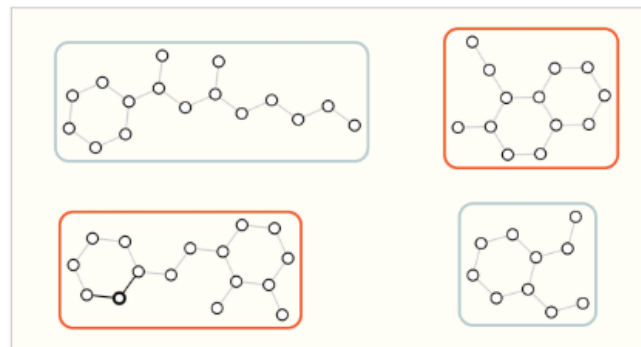
Output Step

- Label 예측
- Task에 따른 구분

Graph Classification



Input: graphs



Output: labels for each graph, (e.g., "does the graph contain two rings?")

그래프 자체가 어떤 레이블에 속하는
지 분류
Full readout

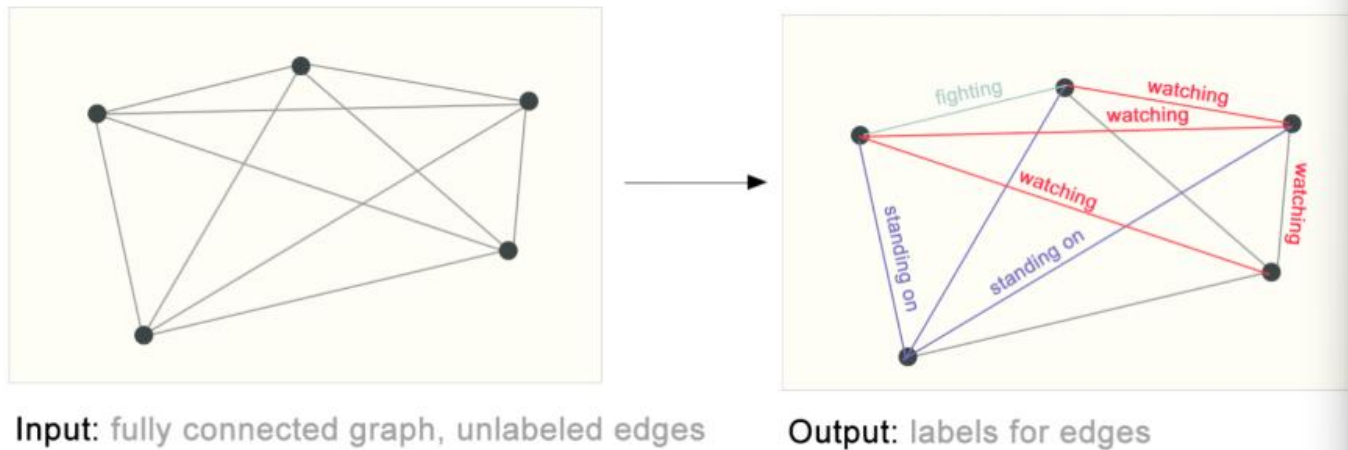
[출처] <https://distill.pub/2021/gnn-intro>, A Gentle Introduction to Graph Neural Networks

$$\text{softmax}\left(\sum_i z_i\right)$$

Output Step

- Label 예측
- Task에 따른 구분

Link Classification



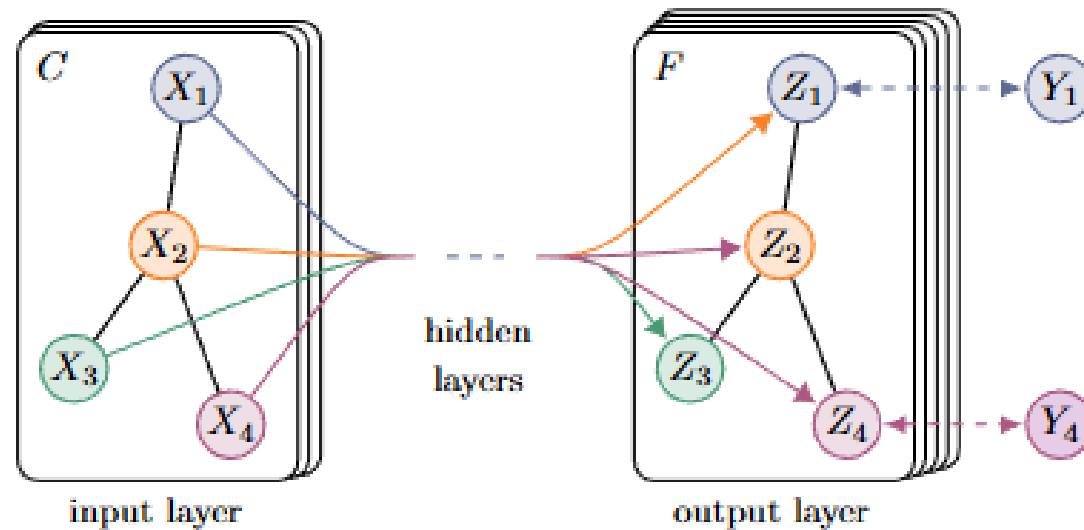
두 정점이 서로 연결될 수 있는지 없는지를 예측

[출처] <https://distill.pub/2021/gnn-intro>, A Gentle Introduction to Graph Neural Networks

$$p(A_{ij}) = \sigma(z_i^T z_j)$$

Graph Convolution Network

- 각 노드별로 계산 그래프를 생성해보자



(a) Graph Convolutional Network

$$Z = f(X, A) = \text{softmax}\left(\hat{A} \text{ReLU}\left(\hat{A}XW^{(0)}\right)W^{(1)}\right)$$

- 손실 함수에 그래프 라플라시안 정규화 항을 추가해서 인접한 노드들이 유사한 레이블을 가질 것이라는 가정을 활용
- 한계 : 노드가 단순히 유사성만을 나타내지 않는데, 표현 능력을 제한할 수 있음

$$\mathcal{L} = \mathcal{L}_0 + \lambda \underset{\text{정규화 항}}{\mathcal{L}_{\text{reg}}} , \quad \text{with} \quad \mathcal{L}_{\text{reg}} = \sum_{i,j} A_{ij} \underbrace{\|f(X_i) - f(X_j)\|^2}_{\text{두 노드 사이의 차이}} = f(X)^{\top} \Delta f(X)$$

Mechanism

1. Spectral Graph Convolution

$$g_{\theta} \star x = U g_{\theta} U^{\top} x$$

- 그래프 푸리에 변환 이용
- 모든 노드에 대해 고유값 분해를 해야 하므로 비용이 비싸다 $O(N^2)$

2. 체비쇼프 다항식을 이용한 근사

$$g_{\theta'} \star x \approx \sum_{k=0}^K \theta'_k T_k(\tilde{L}) x$$

- 체비쇼프 다항식을 도입해 계산 복잡도 낮춤 $O(|E|)$
- K-hop neighborhood된 정보 사용
- 파라미터가 K+1개라서 K가 커지면 복잡해짐

Mechanism

3. 1차 근사

$$g_{\theta'} \star x \approx \theta'_0 x + \theta'_1 (L - I_N) x = \theta'_0 x - \theta'_1 D^{-\frac{1}{2}} A D^{-\frac{1}{2}} x$$

- 체비쇼프 근사를 단순화
- 파라미터가 줄어들었지만 여전히 2개

4. 파라미터 통합

$$g_{\theta} \star x \approx \theta \left(I_N + D^{-\frac{1}{2}} A D^{-\frac{1}{2}} \right) x$$

- 파라미터 1개 -> 단순함
- 여러 레이어를 깊게 쌓았을 때 수치적 불안정성이 발생한 가능성

Mechanism

5. Renormalization Trick

$$Z = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} X \Theta$$

- 수치적 불안정성 해결
- 안정성 (안정적인 고유값 분포)
- 효율성
- $O(|E|FC)O(|E|FC)O(|E|FC)$
- Self-loop
$$\tilde{A} = A + I_N$$
- GCN의 최종적인 형태

Experiment

- Citeseer, Cora, Pubmed, NELL 데이터셋에서의 노드 분류 성능 및 연산 속도
- Semi-supervised (label rate↓)

Table 1: Dataset statistics, as reported in [Yang et al. \(2016\)](#).

Dataset	Type	Nodes	Edges	Classes	Features	Label rate
Citeseer	Citation network	3,327	4,732	6	3,703	0.036
Cora	Citation network	2,708	5,429	7	1,433	0.052
Pubmed	Citation network	19,717	44,338	3	500	0.003
NELL	Knowledge graph	65,755	266,144	210	5,414	0.001

Experiment

- GCN 성능

Method	Citeseer	Cora	Pubmed	NELL
ManiReg [3]	60.1	59.5	70.7	21.8
SemiEmb [28]	59.6	59.0	71.1	26.7
LP [32]	45.3	68.0	63.0	26.5
DeepWalk [22]	43.2	67.2	65.3	58.1
ICA [18]	69.1	75.1	73.9	23.1
Planetoid* [29]	64.7 (26s)	75.7 (13s)	77.2 (25s)	61.9 (185s)
GCN (this paper)	70.3 (7s)	81.5 (4s)	79.0 (38s)	66.0 (48s)
GCN (rand. splits)	67.9 $\pm$ 0.5	80.1 $\pm$ 0.5	78.9 $\pm$ 0.7	58.4 $\pm$ 1.7

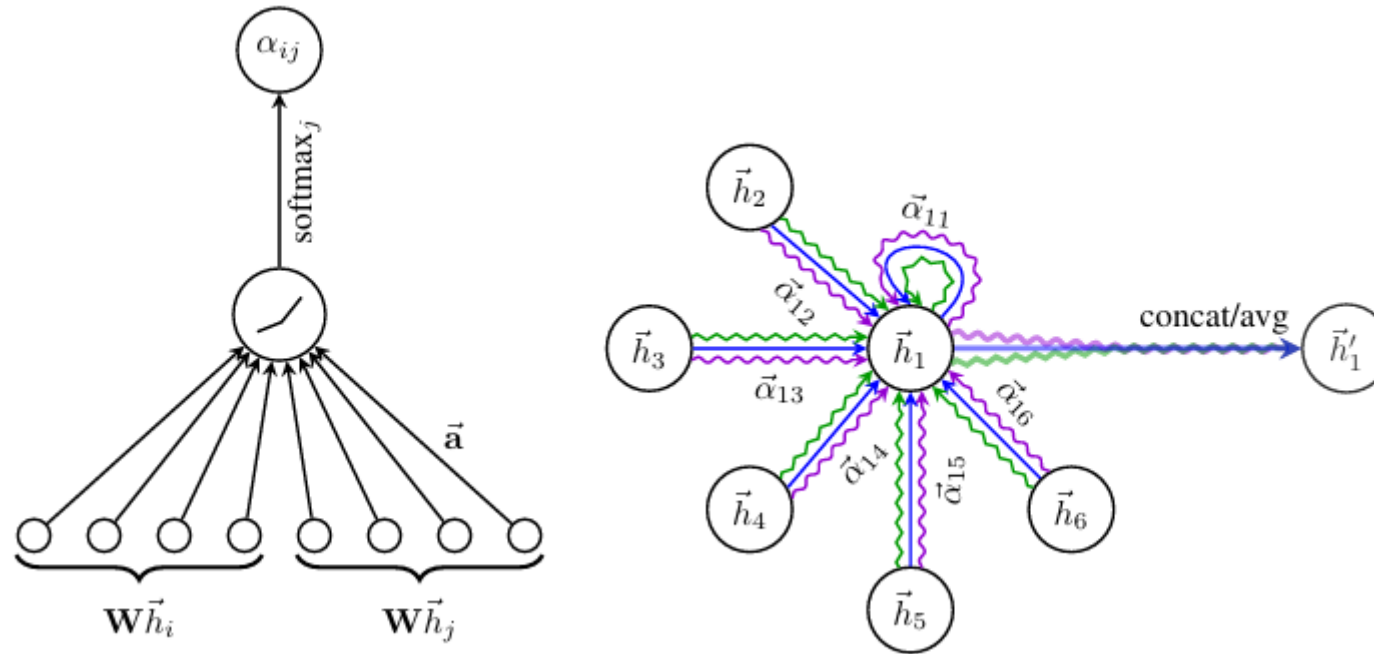
- 전파 모델 평가

Table 3: Comparison of propagation models.

Description	Propagation model	Citeseer	Cora	Pubmed
Chebyshev filter (Eq. 5)	$K = 3$	69.8	79.5	74.4
	$K = 2$	69.6	81.2	73.8
1 st -order model (Eq. 6)	$X\Theta_0 + D^{-\frac{1}{2}}AD^{-\frac{1}{2}}X\Theta_1$	68.3	80.0	77.5
Single parameter (Eq. 7)	$(I_N + D^{-\frac{1}{2}}AD^{-\frac{1}{2}})X\Theta$	69.3	79.2	77.4
Renormalization trick (Eq. 8)	$\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}X\Theta$	70.3	81.5	79.0
1 st -order term only	$D^{-\frac{1}{2}}AD^{-\frac{1}{2}}X\Theta$	68.7	80.5	77.8
Multi-layer perceptron	$X\Theta$	46.5	55.1	71.4

Graph Attention Networks

- 모든 이웃 노드에 동일한 가중치를 부여하는 GCN의 한계를 극복
- Attention Mechanism으로 이웃 노드들의 정보 학습 -> upgrade



Graph Attention Layer

- Self-attention Mechanism 기반
- 기존의 메모리 네트워크와 유사하게 노드의 이웃을 ‘메모리’로 간주하여 이 메모리부터 정보를 attention 하게 가져와서 노드의 특징을 업데이트 함

1. 입력

$$\mathbf{h} = \{\vec{h}_1, \vec{h}_2, \dots, \vec{h}_N\}$$

2. 선형 변환

$$\mathbf{W}\mathbf{h}_i$$

- 차원 맞춤 (원하는 크기의 차원으로 투영시킴)
- 더 고차원적인 특징 추출

Graph Attention Layer

- Self-attention Mechanism 기반
- 기존의 메모리 네트워크와 유사하게 노드의 이웃을 ‘메모리’로 간주하여 이 메모리부터 정보를 attention 하게 가져와서 노드의 특징을 업데이트 함

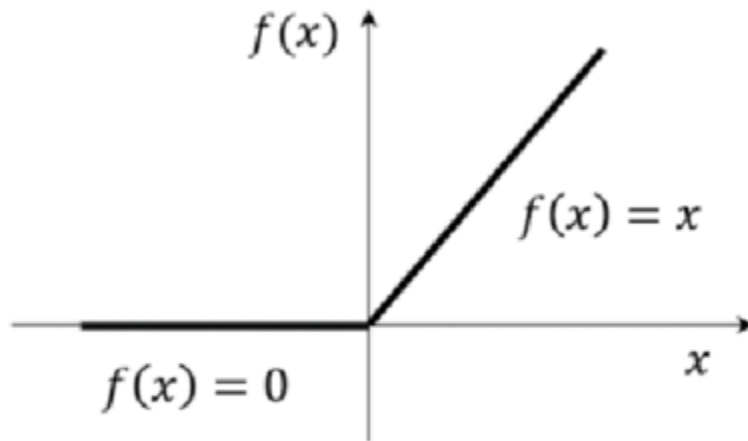
3. self-attention 계수 계산

$$e_{ij} = a(\mathbf{W}\vec{h}_i, \mathbf{W}\vec{h}_j)$$

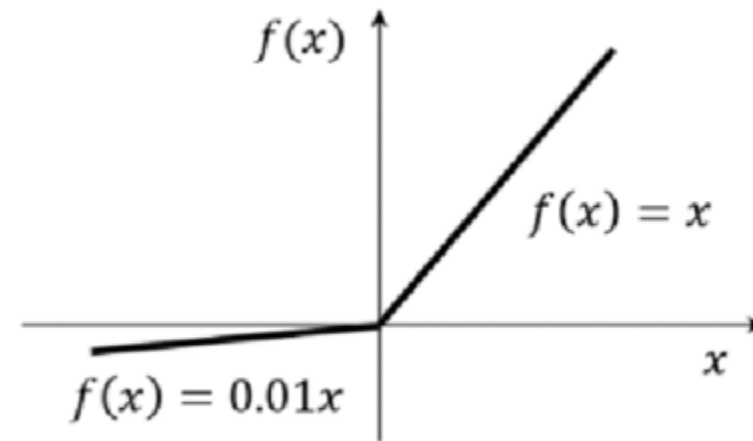
- Attention mechanism을 뜻하는 a 는 LeakyReLU 활성화 함수랑 가중치 벡터 a 를 사용하는 단일 계층 feedforward 신경망으로 구현됨

$$\exp \left(\underset{\text{비선형성 부여}}{\text{LeakyReLU}} \left(\underset{\substack{\text{두 노드의 특징을} \\ \text{concat 하고 내적}}}{\vec{a}^T [\mathbf{W}\vec{h}_i \parallel \mathbf{W}\vec{h}_j]} \right) \right)$$

Graph Attention Layer



ReLU activation function



LeakyReLU activation function

더 세밀한 관계 학습

Graph Attention Layer

- Self-attention Mechanism 기반
- 기존의 메모리 네트워크와 유사하게 노드의 이웃을 ‘메모리’로 간주하여 이 메모리부터 정보를 attention 하게 가져와서 노드의 특징을 업데이트 함

4. Masked Self-Attention

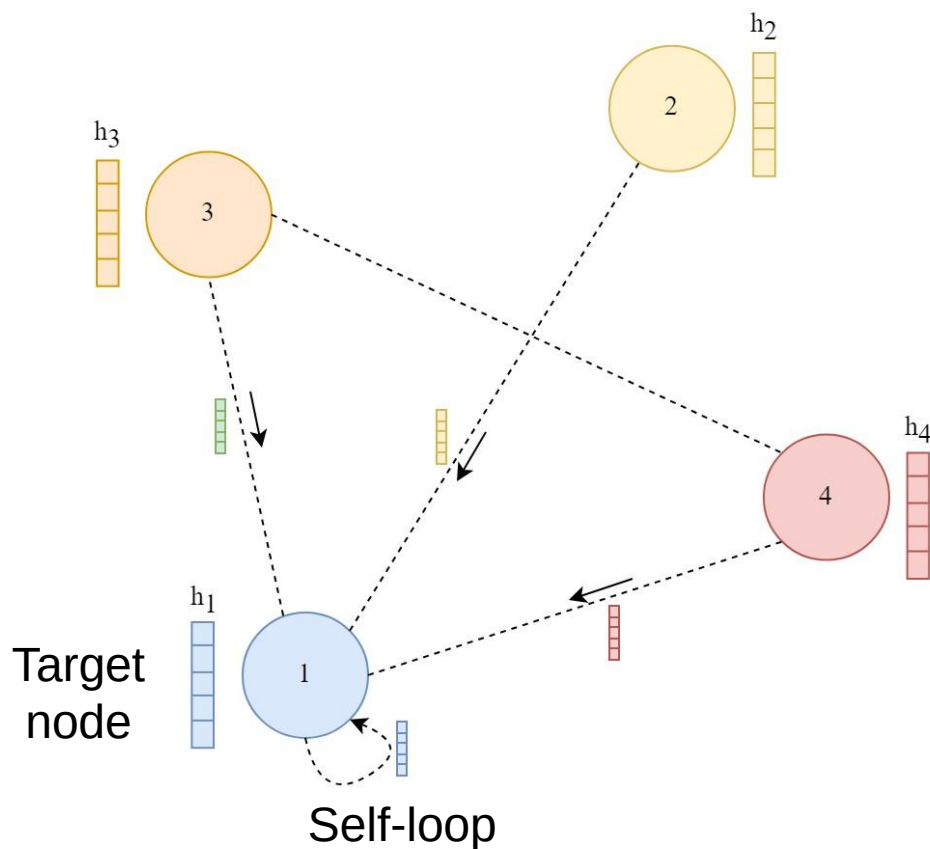
- 이웃 노드에 대해서만 어텐션 계수 계산 (1차)

5. 정규화

$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})} \quad \alpha_{ij} = \frac{\exp \left(\text{LeakyReLU} \left(\vec{a}^T [\mathbf{W} \vec{h}_i \parallel \mathbf{W} \vec{h}_j] \right) \right)}{\sum_{k \in \mathcal{N}_i} \exp \left(\text{LeakyReLU} \left(\vec{a}^T [\mathbf{W} \vec{h}_i \parallel \mathbf{W} \vec{h}_k] \right) \right)}$$

Self-Attention mechanism

Graph Attention Networks



Attention score 계산

$$\text{self-attention}(\mathbf{h}_1) = \begin{cases} \text{attention}(\mathbf{h}_1, \mathbf{h}_2) = \text{Linear}(\mathbf{h}_1, \mathbf{h}_2) \rightarrow \alpha_{12} \\ \text{attention}(\mathbf{h}_1, \mathbf{h}_3) = \text{Linear}(\mathbf{h}_1, \mathbf{h}_3) \rightarrow \alpha_{13} \\ \text{attention}(\mathbf{h}_1, \mathbf{h}_4) = \text{Linear}(\mathbf{h}_1, \mathbf{h}_4) \rightarrow \alpha_{14} \\ \text{attention}(\mathbf{h}_1, \mathbf{h}_1) = \text{Linear}(\mathbf{h}_1, \mathbf{h}_1) \rightarrow \alpha_{11} \end{cases}$$

입력값x중요도

$$\mathbf{h}'_1 = \alpha_{11} \mathbf{h}_1 + \alpha_{12} \mathbf{h}_2 + \alpha_{13} \mathbf{h}_3 + \alpha_{14} \mathbf{h}_4$$

Multi-head attention

- K개의 독립적인 attention mechanism(head)을 각각 적용하여 특징을 추출하고 그 결과들을 연결하거나 평균 내는 방식
- 계산 효율성 – 시간 복잡도 $O(|V|FF' + |E|F')$
- 학습 과정 안정화

concatenation

$$\vec{h}'_i = \parallel_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k \vec{h}_j \right)$$

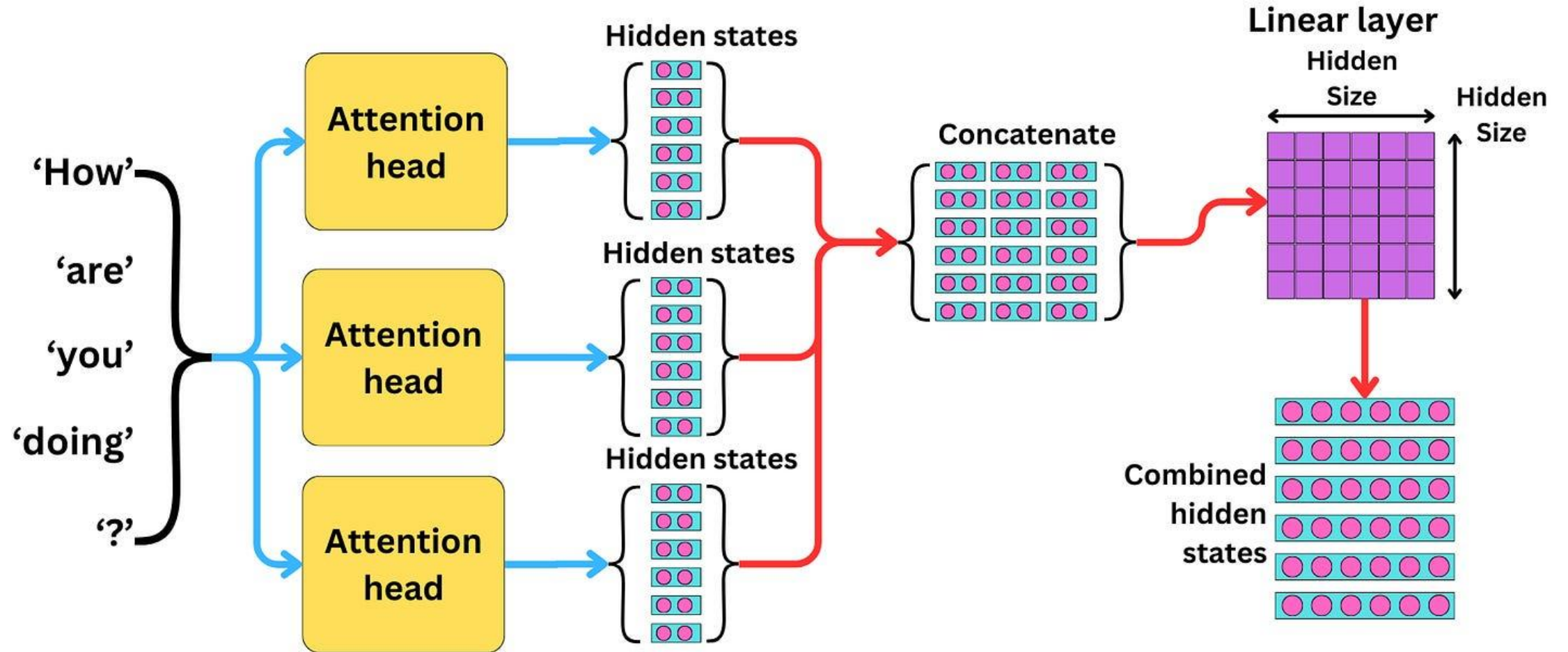
- 모든 head를 손실 없이 다 담고 싶을 때 사용
- 중간 layer에서 정보를 최대한 풍부하게 유지하기 위해 concat 방식 선호

Averaging

$$\vec{h}'_i = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k \vec{h}_j \right)$$

- 최종 결과의 차원(길이)를 원래대로 맞추고 싶을 때 사용
- 마지막 출력층에서 분류를 위한 일정한 벡터가 필요함

Multi-head attention



Experiments

- dataset

Table 1: Summary of the datasets used in our experiments.

	Cora	Citeseer	Pubmed	PPI
Task	Transductive	Transductive	Transductive	Inductive
# Nodes	2708 (1 graph)	3327 (1 graph)	19717 (1 graph)	56944 (24 graphs)
# Edges	5429	4732	44338	818716
# Features/Node	1433	3703	500	50
# Classes	7	6	3	121 (multilabel)
# Training Nodes	140	120	60	44906 (20 graphs)
# Validation Nodes	500	500	500	6514 (2 graphs)
# Test Nodes	1000	1000	1000	5524 (2 graphs)

Experiments

Table 2: Summary of results in terms of classification accuracies, for Cora, Citeseer and Pubmed. GCN-64* corresponds to the best GCN result computing 64 hidden features (using ReLU or ELU).

<i>Transductive</i>			
Method	Cora	Citeseer	Pubmed
MLP	55.1%	46.5%	71.4%
ManiReg (Belkin et al., 2006)	59.5%	60.1%	70.7%
SemiEmb (Weston et al., 2012)	59.0%	59.6%	71.7%
LP (Zhu et al., 2003)	68.0%	45.3%	63.0%
DeepWalk (Perozzi et al., 2014)	67.2%	43.2%	65.3%
ICA (Lu & Getoor, 2003)	75.1%	69.1%	73.9%
Planetoid (Yang et al., 2016)	75.7%	64.7%	77.2%
Chebyshev (Defferrard et al., 2016)	81.2%	69.8%	74.4%
GCN (Kipf & Welling, 2017)	81.5%	70.3%	79.0%
MoNet (Monti et al., 2016)	81.7 $\pm$ 0.5%	—	78.8 $\pm$ 0.3%
GCN-64*	81.4 $\pm$ 0.5%	70.9 $\pm$ 0.5%	79.0 $\pm$ 0.3%
GAT (ours)	83.0 $\pm$ 0.7%	72.5 $\pm$ 0.7%	79.0 $\pm$ 0.3%

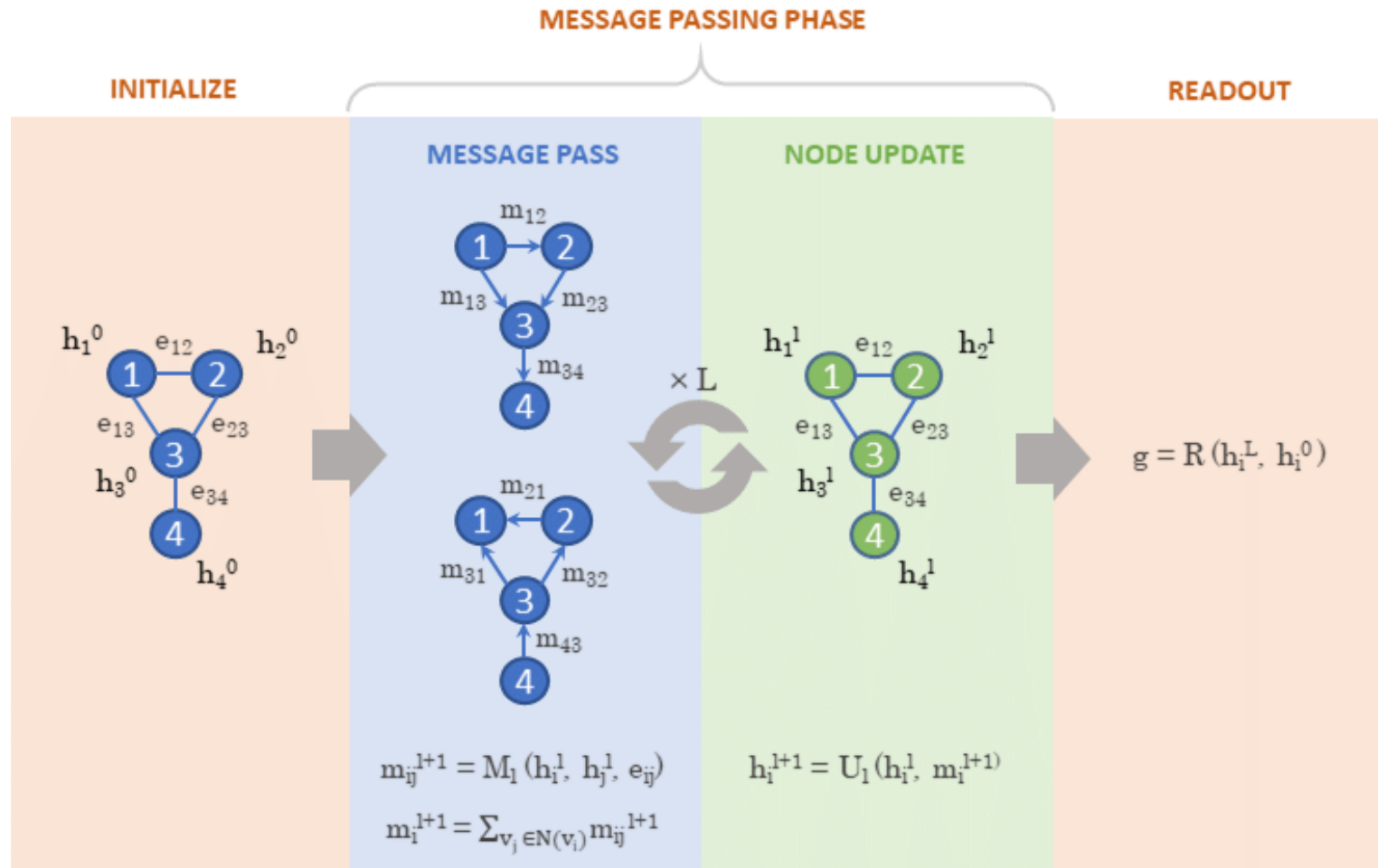
Experiments

- Micro-averaged F1 score

Table 3: Summary of results in terms of micro-averaged F_1 scores, for the PPI dataset. GraphSAGE* corresponds to the best GraphSAGE result we were able to obtain by just modifying its architecture. Const-GAT corresponds to a model with the same architecture as GAT, but with a constant attention mechanism (assigning same importance to each neighbor; GCN-like inductive operator).

<i>Inductive</i>	
Method	PPI
Random	0.396
MLP	0.422
GraphSAGE-GCN (Hamilton et al., 2017)	0.500
GraphSAGE-mean (Hamilton et al., 2017)	0.598
GraphSAGE-LSTM (Hamilton et al., 2017)	0.612
GraphSAGE-pool (Hamilton et al., 2017)	0.600
GraphSAGE*	0.768
Const-GAT (ours)	0.934 ± 0.006
GAT (ours)	0.973 ± 0.002

Message Passing Neural Network



Mechanism

- Message Passing Phase

$$m_v^{t+1} = \sum_{w \in N(v)} M_t(h_v^t, h_w^t, e_{vw})$$

- Message Function
- 노드와 엣지의 정보를 조합해 메시지 생성

$$h_v^{t+1} = U_t(h_v^t, m_v^{t+1})$$

- Vertex Update Function
- 메시지를 집계하여 노드의 상태 업데이트

Mechanism

Matrix Multiplication

$$M(h_v, h_w, e_{vw}) = A_{e_{vw}} h_w$$

- GG-NN 모델에서 사용된 메시지 함수

Edge Network

$$M(h_v, h_w, e_{vw}) = A(e_{vw}) h_w$$

- Edge vector을 입력으로 받아 edge 특징을 더 복잡한 방식으로 모델링해 정교한 학습

Pair Message

$$m_{wv} = f(h_w^t, h_v^t, e_{vw})$$

- 노드 w의 은닉상태와 edge 특징에만 의존하는 것이 아니라, 목적지 노드 v의 은닉 상태까지 고려

Mechanism

- Readout Phase

$$\hat{y} = R(\{h_v^T \mid v \in G\})$$

- Readout Function
- 노드들의 정보를 합쳐 그래프 전체의 특징 추출 (Graph level)

GG-NN의 readout function

$$R = \sum_{v \in V} \sigma \left(i(h_v^{(T)}, h_v^0) \right) \odot \left(j(h_v^{(T)}) \right)$$

- 최종 노드 상태들을 집계하는 방식

Set2set

- MPNN의 마지막 메시지 전달 단계에서 얻어진 모든 노드의 최종 은닉 상태 집합을 입력으로 받음
- 단순히 노드 상태를 합산하는 것보다 더 풍부한 표현력
- 반복적인 계산을 통해 그래프 수준의 임베딩을 생성하며, 최종 예측을 위해 신경망을 통과

Experiments

Table 2. Comparison of Previous Approaches (left) with MPNN baselines (middle) and our methods (right)

Target	BAML	BOB	CM	ECFP4	HDAD	GC	GG-NN	DTNN	enn-s2s	enn-s2s-ens5
mu	4.34	4.23	4.49	4.82	3.34	0.70	1.22	-	0.30	0.20
alpha	3.01	2.98	4.33	34.54	1.75	2.27	1.55	-	0.92	0.68
HOMO	2.20	2.20	3.09	2.89	1.54	1.18	1.17	-	0.99	0.74
LUMO	2.76	2.74	4.26	3.10	1.96	1.10	1.08	-	0.87	0.65
gap	3.28	3.41	5.32	3.86	2.49	1.78	1.70	-	1.60	1.23
R2	3.25	0.80	2.83	90.68	1.35	4.73	3.99	-	0.15	0.14
ZPVE	3.31	3.40	4.80	241.58	1.91	9.75	2.52	-	1.27	1.10
U0	1.21	1.43	2.98	85.01	0.58	3.02	0.83	-	0.45	0.33
U	1.22	1.44	2.99	85.59	0.59	3.16	0.86	-	0.45	0.34
H	1.22	1.44	2.99	86.21	0.59	3.19	0.81	-	0.39	0.30
G	1.20	1.42	2.97	78.36	0.59	2.95	0.78	.84 ²	0.44	0.34
Cv	1.64	1.83	2.36	30.29	0.88	1.45	1.19	-	0.80	0.62
Omega	0.27	0.35	1.32	1.47	0.34	0.32	0.53	-	0.19	0.15
MAE Average	2.17	2.08	3.37	53.97	1.35	2.59	1.36	-	0.68	0.52

Experiments

MAE Error Ratio

Table 3. Models Trained Without Spatial Information

Model	Average Error Ratio
GG-NN	3.47
GG-NN + Virtual Edge	2.90
GG-NN + Master Node	2.62
GG-NN + set2set	2.57

GGNN towers 변형

Table 4. Towers vs Vanilla GG-NN (no explicit hydrogen)

Model	Average Error Ratio
GG-NN + joint training	1.92
towers8 + joint training	1.75
GG-NN + individual training	1.53
towers8 + individual training	1.37

